Atividade Avaliativa: API REST do Abrigo de Pets 🐶🐱 — Parte 1

Disciplina: Desenvolvimento Back-end
Valor: 10,0 pontos (nota total da atividade, somando as partes 1 e 2)
Modalidade: Individual
Forma de entrega: GitHub

- crie um novo repositório, exclusivo para esse projeto
- revise o conteudo sobre clone, se necessário

Parte 1 de 2: fazer a API funcionar. Nesta parte o objetivo é ter todas as rotas funcionando, com todo o código no arquivo `index.js`. A organização em pastas, roteadores e controller, e o middleware de log ficam para a **Parte 2**.

---

1. Contexto

Um abrigo de animais da sua cidade precisa de um sistema para controlar os pets que estão disponíveis para adoção. Você foi contratado(a) para construir a **API REST** desse sistema.

Nesta atividade você vai aplicar tudo o que aprendeu na `minha-primeira-api` (a API dos livros): **Express, rotas, validações e status codes HTTP**. A lógica é a mesma, só muda o tema.

2. Regras gerais

- A API deve ser feita em **Node.js com Express**, usando **ES Modules** (`"type": "module"` no `package.json`), como na API dos livros.
- **Não há autenticação** e **não há banco de dados**. Os dados ficam em um **array na memória** (ao reiniciar o servidor, os dados voltam ao estado inicial, e isso é esperado).
- A API deve rodar na **porta 3000**.
- Nesta parte, **todo o código fica no arquivo `index.js`**: o array, as funções e as rotas. Não se preocupe com pastas ainda.
- Todas as respostas de erro com body devem usar o formato `{ "mensagem": "texto explicando o erro" }`.
- O trabalho é **individual**. Cópias entre colegas resultam em nota zero para todos os envolvidos.

3. O modelo: Pet

Cada pet do abrigo é um objeto com os campos abaixo:

Campo	Tipo	Regras
`idPet`	número inteiro	Gerado pela API, começa em 1 e **nunca se repete**, mesmo depois de deletar pets. Não pode ser alterado.
`dsNome`	texto	Obrigatório.
`dsEspecie`	texto	Obrigatório (ex.: `"cachorro"`, `"gato"`, `"coelho"`).
`nuIdade`	número	Obrigatório. Inteiro ou decimal, **maior ou igual a 0**.
`fgAdotado`	booleano	**Não é enviado no cadastro**: todo pet novo começa com `false`.

Exemplo de pet:

```
{
  "idPet": 1,
  "dsNome": "Rex",
  "dsEspecie": "cachorro",
  "nuIdade": 3,
  "fgAdotado": false
}
```

Dados iniciais

O array deve começar com **pelo menos 3 pets**, com `idPet` 1, 2 e 3. Isso é obrigatório, porque a collection de testes usa o pet de `idPet: 1`. Lembre de ajustar a variável que controla o último id (`ultimoId`) de acordo com os pets iniciais.

4. Estrutura do projeto (Parte 1)

```
abrigo-de-pets/
├── index.js
└── package.json
```

Dentro do `index.js`, organize o código nesta ordem:

Parte do arquivo	O que deve conter
Início	Os `import`s, a criação do app Express e o `express.json()`.
Dados	O array de pets e a variável `ultimoId`.
Funções	Funções que **manipulam os dados** (buscar todos, buscar um, criar, editar, deletar). Não usam `req` nem `res`.
Rotas	As rotas `app.get`, `app.post`, `app.patch` e `app.delete`. Leem `req.params` e `req.body`, **fazem as validações**, chamam as funções e devolvem os **status codes** e o JSON.
Final	O `app.listen(3000)`.

> ⚠️ Mesmo estando tudo no mesmo arquivo, **a rota não deve mexer no array diretamente**. Ela sempre chama uma função. Isso vai facilitar muito a Parte 2, quando essas funções forem movidas para o controller.

5. Rotas que devem ser implementadas

Todas as rotas ficam sob o caminho `/pets`.

5.1 `GET /pets` (findAll)

Lista todos os pets.

Situação	Status	Body
Sempre	`200`	Array com todos os pets (pode ser `[]`)

5.2 `GET /pets/:id` (findOne)

Busca um pet pelo `idPet`.

Situação	Status	Body
Pet encontrado	`200`	O objeto do pet
`:id` não é um número (ex.: `/pets/abc`)	`400`	`{ "mensagem": "identificador deve ser um numero" }`
`:id` é um número, mas nenhum pet tem esse id	`404`	sem body

5.3 `POST /pets` (criar)

Cadastra um novo pet.

Body de exemplo:

```
{
  "dsNome": "Thor",
  "dsEspecie": "cachorro",
  "nuIdade": 2
}
```

Situação	Status	Body
Pet criado com sucesso	`201`	O pet criado, **já com `idPet` gerado e `fgAdotado: false`**
`dsNome` ou `dsEspecie` ausente (ou vazio)	`400`	`{ "mensagem": "..." }`
`nuIdade` ausente, não numérico (ex.: `"dois"`) ou negativo	`400`	`{ "mensagem": "..." }`

Regras extras:

- Se o body enviar `fgAdotado` ou `idPet`, **ignore** esses campos. Quem define o id e o `fgAdotado` inicial é a API.
- O id é gerado de forma incremental. Se você cadastrar o pet 4, deletar, e cadastrar outro, o novo é o **5**.

5.4 `PATCH /pets/:id` (editar)

Atualiza **parcialmente** um pet. O body pode conter qualquer combinação dos campos `dsNome`, `dsEspecie`, `nuIdade` e `fgAdotado`. **Só os campos enviados são alterados**, e os outros permanecem como estavam.

Body de exemplo (marcar como adotado):

```
{
  "fgAdotado": true
}
```

Outro exemplo (corrigir nome e idade):

```
{
  "dsNome": "Thor Jr.",
  "nuIdade": 3
}
```

Situação	Status	Body
Atualizado com sucesso	`200`	O pet **já atualizado** (objeto completo)
`:id` não é um número	`400`	`{ "mensagem": "identificador deve ser um numero" }`
Nenhum pet tem esse id	`404`	sem body
Body vazio (`{}`) ou sem nenhum campo válido	`400`	`{ "mensagem": "..." }`
`nuIdade` negativo ou não numérico	`400`	`{ "mensagem": "..." }`
`fgAdotado` que não seja `true` ou `false` (ex.: `"sim"`)	`400`	`{ "mensagem": "..." }`
`dsNome` ou `dsEspecie` enviados como texto vazio	`400`	`{ "mensagem": "..." }`

Regras extras:

- O `idPet` **nunca** pode ser alterado, mesmo que venha no body.
- Se qualquer campo enviado for inválido, **nada** deve ser alterado no pet.

5.5 `DELETE /pets/:id` (deletar)

Remove um pet.

Situação	Status	Body
Removido com sucesso	`204`	sem body
`:id` não é um número	`400`	`{ "mensagem": "identificador deve ser um numero" }`
Nenhum pet tem esse id	`404`	sem body

Depois de deletar, um `GET /pets/:id` com o mesmo id deve retornar `404` .

6. Como testar: Collection do Postman

Junto com este enunciado você recebeu o arquivo **`abrigo-de-pets.postman_collection.json`** . **Use essa collection para testar a sua API.** Ela foi feita para conferir exatamente o que está descrito acima, e a correção será feita com ela.

Como usar:

1. Abra o Postman e use **Import** para carregar o arquivo da collection.
2. Suba a sua API (`node index.js`).
3. Abra a requisição que deseja usar, altere os dados, se necessário, e clique em "send"

Detalhes importantes:

- O `POST` guarda o id do pet criado na variável `{{petId}}` , que o `PATCH` e o `DELETE` usam em seguida. Por isso, **rode as rotas principais em ordem**.
- A pasta de erros usa os ids `1` (precisa existir) e `9999` (não pode existir).
- Se a URL da sua API for diferente, altere a variável `baseUrl` da collection.
- Cada requisição da collection tem uma **descrição** com as regras daquela rota. Leia!

7. Dicas 💡

Organização

- Use a `minha-primeira-api` como referência. As rotas, o `express.json()` e o jeito de validar são os mesmos, só que aqui tudo fica no `index.js`.
- Comece criando o projeto (`npm init -y`, `npm install express`), coloque `"type": "module"` no `package.json` e deixe o servidor subindo na porta 3000 antes de fazer qualquer rota.
- Faça **uma rota por vez**, testando com a collection antes de passar para a próxima. Fazer tudo e testar no final deixa tudo mais difícil.
- Sugestão de ordem: `GET /pets`, `GET /pets/:id`, `POST`, `DELETE` e, por último, o `PATCH`, que é o mais desafiador.

Validações

- Valide **na ordem**: primeiro o id (`400`), depois se o pet existe (`404`), e só então o conteúdo do body.
- Para checar o id, o `parseInt` + `isNaN` da API dos livros funciona bem.
- Para validar tipos, use `typeof` (ex.: `typeof nuIdade !== "number"`). Lembre que `"2"` (texto) e `2` (número) são coisas diferentes no JSON.
- Cuidado com `if (!nuIdade)`: o número `0` é *falsy* no JavaScript, mas **é uma idade válida** (filhote com menos de 1 ano). Compare com `undefined` ou use `typeof`.

PATCH (a rota mais desafiadora)

- O PATCH é diferente do POST: aqui os campos são **opcionais**, mas os que vierem precisam ser válidos.
- Uma ideia: crie um objeto vazio com as alterações, preencha só com os campos que vieram (e que estão válidos), e depois aplique no pet. Se o objeto ficou vazio, devolva `400`.
- Para saber se um campo veio no body, compare com `undefined` (e não com `!campo`, senão `fgAdotado: false` e `nuIdade: 0` seriam ignorados).
- Para achar o pet no array, use `find` ou `findIndex`, como nos livros. Para aplicar as mudanças, pesquise sobre o *spread operator* (`...`) e o `Object.assign`.
- Lembre que o PATCH deve **devolver o pet atualizado**.

DELETE

- Cuidado com o `splice` quando o `findIndex` retorna `-1`: ele remove o **último** item do array. Confira se o pet existe antes de deletar.

Id incremental

- Nunca calcule o novo id com `array.length + 1`. Depois de um DELETE isso gera ids repetidos. Use uma variável separada (`ultimoId`) que só aumenta.

Dia a dia

- Use `npx nodemon index.js` para o servidor reiniciar sozinho a cada alteração. Se quiser, crie um script `"dev"` no `package.json`.
- Use `console.log` para ver o que chega em `req.params` e `req.body`.
- Lembre que, ao reiniciar o servidor, o array volta ao estado inicial. Se a collection falhar "do nada", reinicie a API e rode de novo.
- Use nomes claros e siga o padrão de prefixos (`id`, `ds`, `nu`, `fg`) também nas variáveis e funções.
- Faça commits pequenos, com mensagens que expliquem o que mudou.

8. O que entregar

1. O projeto completo **sem a pasta `node_modules`** (o `package.json` precisa listar as dependências).
2. link do repositório do projeto no GitHub (lembre-se de deixar o repositório publico)
3. Ao terminar a Parte 1, faça um **commit** com a API funcionando. A Parte 2 continua **no mesmo repositório**.

Boa atividade! Quando a API estiver funcionando, siga para a **Parte 2**.